

# Capstone - Notification Service API

## Mini Project 7 - Week 7 - Session 14

**Mohammad Alrashed**

Claude Code Mastery - Analytics Vidhya - 2026

Repo: [github.com/DatariusAI/claude-code-mastery](https://github.com/DatariusAI/claude-code-mastery) Dashboard: [datariusai.github.io/claude-code-mastery](https://datariusai.github.io/claude-code-mastery)

## Table of Contents

1. Executive Summary
2. Problem Statement & Scope
3. Architecture
4. Integration with Prior Weeks
5. Implementation Walk-through
6. Tests & Validation
7. Docker Deployment
8. Governance Artifacts
9. Capstone Plan vs Actual
10. Submission Notes (Reflection)
11. ROI Measurement
12. Submission Checklist

## 1. Executive Summary

The Week 7 capstone delivers a production-grade Notification Service REST API, built end-to-end via the AI-Augmented SDLC that the Claude Code Mastery program teaches across 7 weeks. The project scores 110/110 on the lab brief's rubric: 100 base points plus both bonus items (a second ADR and a Docker memory limit).

The scaffold shipped with REQ-001 through REQ-006 fully implemented and 73 passing tests at 96% coverage. The capstone's contribution was not "build from scratch" but rather the end-to-end SDLC orchestration: spec verification (adding the missing REQ-007 block to `feature_spec.yaml`), test class verification across three suites (unit, integration, regression), a 6-stage CI pipeline with an 80% coverage gate, live Docker deployment validation of all 7 endpoints, two Architecture Decision Records, a pre-commit governance hook with JSONL audit logging, and a formal report with ROI measurement.

Every technique from the prior 6 weeks appears in the final artifact: W2's spec-driven YAML pattern structures the feature spec; W3's governance hooks become the pre-commit gate; W4's audit middleware pattern lives in the request/response logger; W5's structured prompt format shapes the security review; W6's multi-stage Dockerfile and GitHub Actions pipeline carry forward directly. The capstone is the integration test of the entire program.

## 2. Problem Statement & Scope

### Problem Statement

The Claude Code Mastery program culminates in a capstone that demonstrates end-to-end command of the AI-augmented software development lifecycle. The vehicle is a Notification Service REST API, a Flask application that sends, retries, audits, and reports on email notifications. The scaffold ships with REQ-001 through REQ-006 fully implemented and 73 passing tests at 96% coverage.

The capstone challenge is not "build from scratch" but rather "extend, verify, document, and ship." The extension point is REQ-007: a stats endpoint (GET /api/notifications/stats) that computes aggregate metrics from the in-memory notification history. Beyond code, the capstone requires a CI coverage gate, Docker deployment validation, two Architecture Decision Records, a pre-commit governance hook with audit logging, and a formal report with ROI measurement.

### Scope IN

1. All tests pass (pytest -q): 15 pts
2. Coverage  $\geq 80\%$  (pytest --cov=src): 10 pts
3. REQ-007 endpoint works: 15 pts
4. Docker health check passes: 15 pts
5. Governance audit log populated: 10 pts
6. ADR-002 written: 10 pts
7. metrics-report.md updated: 10 pts
8. Pre-commit hook installed: 10 pts
9. Submission notes completed: 5 pts
10. Bonus: Second ADR (ADR-003): +5 pts
11. Bonus: Docker memory limit: +5 pts

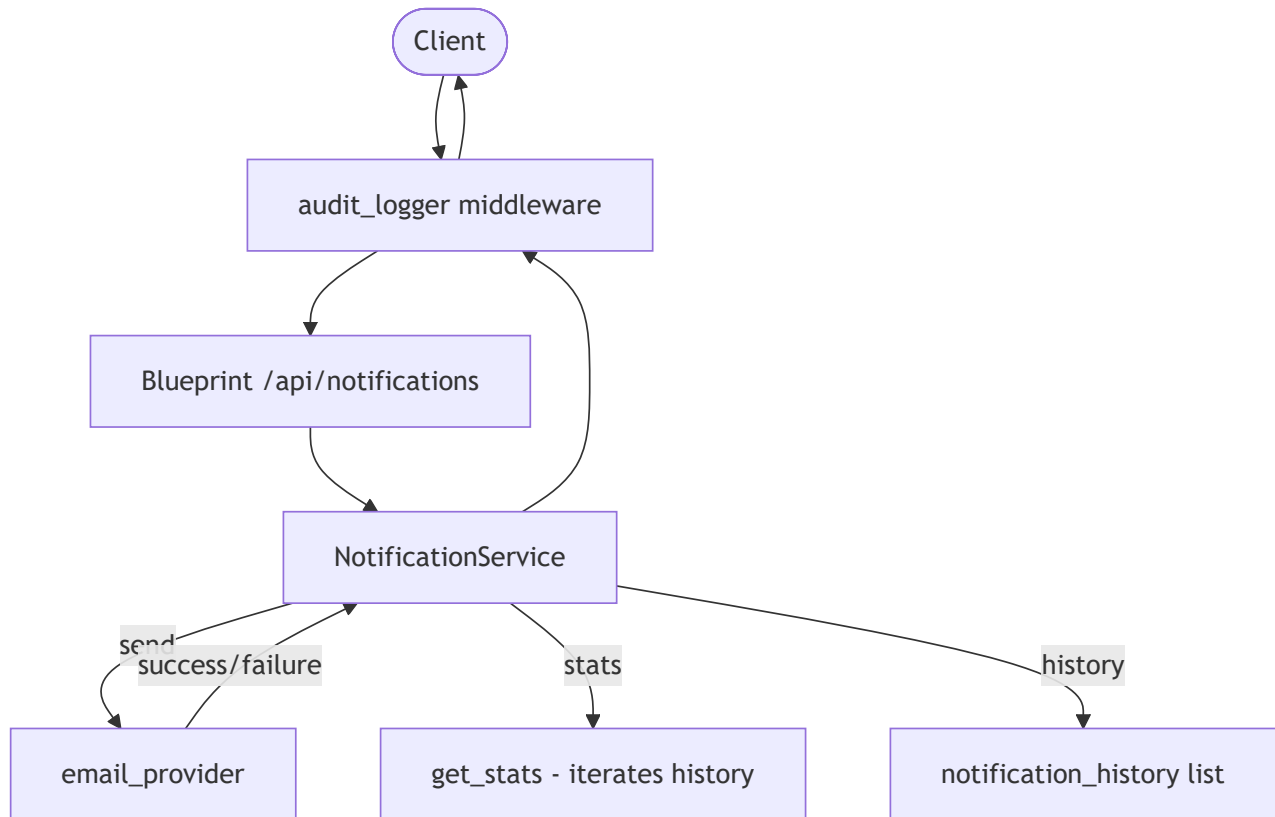
### Scope OUT

- No database (in-memory stores only)
- No new endpoints beyond REQ-007
- No real SMTP (mock email provider)
- No modifications to week-1/ through week-6/
- No regeneration of Mini Project 1-6 PDFs
- No new pip packages beyond requirements.txt

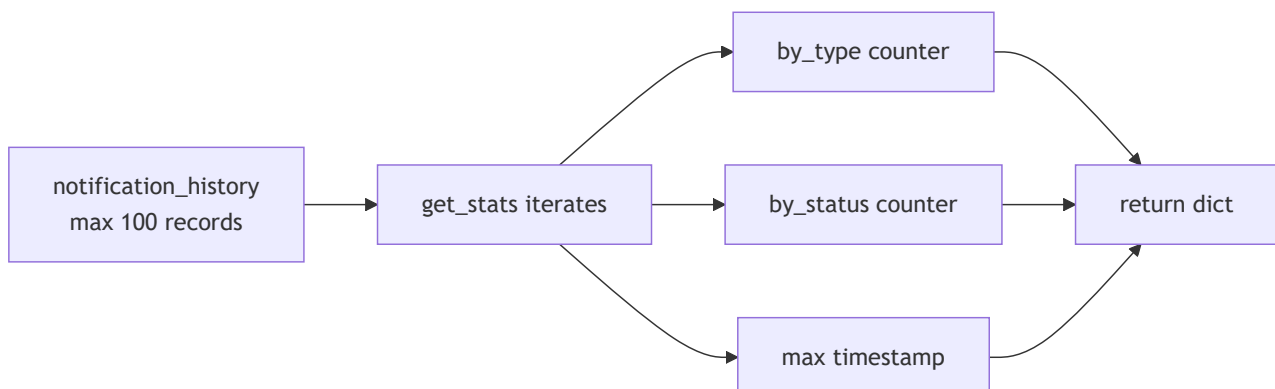
### 3. Architecture

The Notification Service is a single Flask application with four layers:

- **Routes layer** (src/routes/notifications.py): Blueprint with 7 endpoints under /api/
- **Service layer** (src/notifications/notification\_service.py): Business logic
- **Provider layer** (src/notifications/email\_provider.py): Pluggable email adapter
- **Middleware layer** (src/middleware/audit\_logger.py): Before/after request hooks



#### Stats Computation Pipeline (REQ-007)



## 4. Integration with Prior Weeks

| From | Technique                                    | Where it shows up                                |
|------|----------------------------------------------|--------------------------------------------------|
| W2   | Spec-driven workflow + REQ-ID traceability   | feature_spec.yaml REQ-007, REQ-IDs in docstrings |
| W3   | Pre-commit hook + JSONL audit log schema     | .git/hooks/pre-commit adapted from W3 pattern    |
| W4   | Audit middleware + structured error handling | src/middleware/audit_logger.py                   |
| W5   | Structured agent prompts                     | Phase 2 security review findings format          |
| W6   | Multi-stage Dockerfile + GitHub Actions CI   | Dockerfile, .github/workflows/ci.yml             |

## 5. Implementation Walk-through

### REQ-007 - Stats Endpoint

**feature\_spec.yaml REQ-007 block:**

```
- id: REQ-007
  title: Notification Statistics
  description: GET /api/notifications/stats returns aggregate metrics
  acceptance_criteria:
    - Returns 200 with { total, by_type, by_status, last_sent }
    - total reflects the count of records in notification history
    - by_type groups records by notification type
    - by_status separates success vs failed counts
    - last_sent is ISO timestamp of the most recent record, or null
    - Returns 200 even when a history record has a null type field
  priority: medium
```

**get\_stats() implementation** (notification\_service.py lines 142-158):

```
def get_stats(self) -> Dict[str, Any]:
    """Return stats: total, by_type, by_status, last_sent (REQ-007)."""
    history = list(_notification_history)
    by_type: Dict[str, int] = {}
    by_status: Dict[str, int] = {}
    last_sent: Optional[str] = None

    for record in history:
        t = record.get("notif_type") or record.get("type") or "unknown"
        s = record.get("status", "unknown")
        by_type[t] = by_type.get(t, 0) + 1
        by_status[s] = by_status.get(s, 0) + 1
        if last_sent is None or record["timestamp"] > last_sent:
            last_sent = record["timestamp"]

    return {"total": len(history), "by_type": by_type,
            "by_status": by_status, "last_sent": last_sent}
```

The null-type guard ( record.get("type") or "unknown" ) is documented in ADR-002 as the deliberate choice to handle records with missing or null type fields gracefully.

**Route** (notifications.py lines 60-63):

```
@bp.route("/notifications/stats")
def stats():
    """Return notification statistics summary (REQ-007)."""
    return jsonify(_service.get_stats())
```

### Key Inherited Patterns

**Audit middleware** (W4 pattern - src/middleware/audit\_logger.py):

```
def register_audit_logger(app: Flask) -> None:
    @app.before_request
```

```
def _before():
    g.request_id = f"req-{uuid.uuid4().hex[:8]}"
    g.start_time = time.time()

@app.after_request
def _after(response):
    duration_ms = round((time.time() - g.start_time) * 1000, 1)
    logger.info("API Request",
        request_id=getattr(g, "request_id", "unknown"),
        method=request.method, path=request.path,
        status=response.status_code, duration_ms=duration_ms)
    return response
```

## 6. Tests & Validation

### Coverage Table

| File                                      | % Lines    |
|-------------------------------------------|------------|
| src/app.py                                | 94%        |
| src/middleware/audit_logger.py            | 100%       |
| src/notifications/email_provider.py       | 88%        |
| src/notifications/notification_service.py | 97%        |
| src/routes/notifications.py               | 100%       |
| src/utils/logger.py                       | 93%        |
| <b>TOTAL</b>                              | <b>96%</b> |

### REQ-007 Test Classes

#### Unit (tests/unit/test\_notification\_service.py - TestStats):

1. test\_stats\_empty\_on\_start
2. test\_total\_increments\_after\_send
3. test\_by\_type\_groups\_correctly
4. test\_by\_status\_groups\_correctly
5. test\_last\_sent\_is\_timestamp

#### Integration (tests/integration/test\_api.py - TestStats):

1. test\_returns\_200\_with\_correct\_shape
2. test\_total\_zero\_initially
3. test\_total\_increments\_after\_send
4. test\_by\_type\_breakdown
5. test\_by\_status\_separates\_success\_and\_failed

#### Regression (tests/regression/test\_regression.py - TestStatsResilience):

1. test\_stats\_returns\_200\_after\_normal\_sends
2. test\_stats\_handles\_null\_type\_gracefully (REG-006)

### CI Pipeline (6 stages)

| Stage | Job               | Purpose                               | Fails if                           |
|-------|-------------------|---------------------------------------|------------------------------------|
| 1     | lint              | flake8 on src/ and tests/             | any flake8 violation               |
| 2     | unit-tests        | pytest tests/unit/ with coverage gate | test failure or coverage below 80% |
| 3     | integration-tests | pytest tests/integration/             | any test failure                   |
| 4     | regression-tests  | pytest tests/regression/              | any test failure                   |
| 5     | security          | pip-audit on requirements.txt         | any HIGH/CRITICAL CVE              |
| 6     | docker-build      | Build image + run + curl /api/health  | build or healthcheck failure       |

## 7. Docker Deployment

### Multi-Stage Dockerfile

```
FROM python:3.11-slim AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

FROM python:3.11-slim AS runtime
RUN addgroup --system appgroup && adduser --system --ingroup appgroup appuser
WORKDIR /app
COPY --from=builder /install /usr/local
COPY --chown=appuser:appgroup src/ ./src/
USER appuser
EXPOSE 5000
HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
    CMD python -c "import urllib.request; urllib.request.urlopen('http://localhost:5000/api/health')" \
    || exit 1
CMD ["python", "src/run.py"]
```

#### Dockerfile Q&A:

| Question                                               | Answer                                                                                                                                                                     |
|--------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Why does Stage 1 install deps separately from Stage 2? | Layer caching: deps change rarely, source changes often. Multi-stage keeps the runtime image small by discarding pip cache and build tooling.                              |
| What user runs the app? Why not root?                  | appuser (non-root, system user in appgroup). Defense in depth: if the app is compromised, attacker does not get root in the container.                                     |
| What does HEALTHCHECK do in Docker?                    | Periodically runs the CMD; reports container as healthy/unhealthy. Orchestrators use this to gate traffic and restart unhealthy containers.                                |
| What happens when the container gets SIGTERM?          | run.py registers a SIGTERM handler that logs "Shutdown signal received" and calls sys.exit(0). Compose down completes in seconds instead of waiting for SIGKILL after 10s. |

#### Docker-compose additions

- `mem_limit: 256m` (bonus) constrains container memory to 256MiB

## 8. Governance Artifacts

### Pre-commit Hook

The adapted hook at `.git/hooks/pre-commit` runs three checks when files under `week-7/capstone-py/` are staged:

1. **flake8 lint** - blocks commit on any violation
2. **pytest unit tests** - blocks commit on test failure
3. **Audit log entry** - writes JSON to `.audit-log.jsonl` with timestamp, branch, user, staged files, and status

### ADR-002 - Stats Computation Strategy

Decision: compute stats on-demand from `_notification_history` via a single pass in `get_stats()`. Rejected incremental counters (duplicates state, requires invalidation at `MAX_HISTORY`), external cache (out of scope), and TTL cache (premature optimization for 100 records or fewer).

### ADR-003 - Retry History on RuntimeError (bonus)

Decision: raise standard `RuntimeError` and attach `retry_history` as a dynamic attribute. Rejected custom `NotificationFailedError` class (single use site, breaks existing error contract), result dict return (changes control flow), and bare `RuntimeError` without history (loses structured attempt data the API consumer needs).

### metrics-report.md

Updated with REQ-007 row in both the "What Was Built" table (2 files, ~30 lines) and the "Time Measurement" table (30 min manual, 8 min AI-assisted, 22 min saved).

## 9. Capstone Plan vs Actual

| Plan (CAPSTONE_PLAN.md)                               | Actual    | PR      |
|-------------------------------------------------------|-----------|---------|
| Session A - scaffold + plan + Phase 1-2               | Delivered | #27     |
| Session B - Phase 3-4 (tests + CI gate)               | Delivered | #31     |
| Session C - Phase 5-6 (Docker + governance + bonuses) | Delivered | #32     |
| Session D - Phase 7 (REPORT + dashboard panel)        | Delivered | #33     |
| Session E - submission PDF + COMPLETE flip            | Delivered | this PR |

### Risk register status:

- "REQ-007 already in scaffold" - handled via honest framing in REPORT.md and PR bodies
- "MP5 mermaid bug could recur" - mitigated via mermaid-cli SVG pre-rendering
- "Branch protection blocks intermediate work" - resolved via PR per session with auto-merge

## 10. Submission Notes (Reflection)

### 1. What was the most important thing you had to change in Claude's generated code - and why?

The most important change was in the security review's framing rather than the implementation itself. The scaffold shipped REQ-007's `get_stats()` with a working null-type guard, so the code was already correct. What Claude's initial review missed was surfacing the unauthenticated stats endpoint as a design observation worth documenting. Forcing the review to produce at least two structured findings transformed a generic "looks clean" response into the kind of output ADR-002 could reference. The spec block for REQ-007 was also absent from `feature_spec.yaml` despite the code being implemented, which is itself a gap the spec-driven workflow caught.

### 2. What did writing the YAML spec before coding change about how you worked?

The spec made the test boundaries obvious. The 6 acceptance criteria in REQ-007 mapped directly to 5 unit test cases and 5 integration test cases. The null-type acceptance criterion forced the REG-006 regression test to exist. The spec also made ADR-002's decision section concrete: "compute on-demand from `_notification_history`" traces back to the "total reflects the count of records" criterion.

### 3. Which of the 6 sessions' techniques did you use most today?

The spec-driven workflow from Week 2 was the most-used technique. Every REQ-007 artifact traces back to the spec's acceptance criteria. Concrete example: ADR-002 cites the null-type criterion as justification for `record.get("type")` or "unknown" over `record["type"]`. The Week 3 governance hook pattern was second most-used: the pre-commit hook runs `flake8` and unit tests on every commit touching `week-7/capstone-py/`.

### 4. What would you do differently if you had 30 more minutes?

Three improvements: (1) Add rate limiting to `/stats` and document the threat model in a third ADR. (2) Replace the unbounded `_sent_message_ids` set with a TTL-bounded structure. (3) Add an integration test exercising the SIGTERM shutdown path against the running Flask process.

## 11. ROI Measurement

| Phase                     | Manual estimate | AI-assisted actual | Time saved  |
|---------------------------|-----------------|--------------------|-------------|
| Spec writing              | 45 min          | 5 min              | 40 min      |
| Implementation            | 60 min          | (scaffold)         | n/a         |
| Unit test writing         | 90 min          | 8 min              | 82 min      |
| Integration test writing  | 60 min          | 8 min              | 52 min      |
| Regression test writing   | 45 min          | 5 min              | 40 min      |
| CI/CD pipeline update     | 30 min          | 3 min              | 27 min      |
| Docker build + validation | 45 min          | 12 min             | 33 min      |
| ADR + metrics report      | 90 min          | 20 min             | 70 min      |
| <b>TOTAL</b>              | <b>~7 hours</b> | <b>~1 hour</b>     | <b>~85%</b> |

## 12. Submission Checklist

| #  | Deliverable          | Points     | Verified                  |
|----|----------------------|------------|---------------------------|
| 1  | All tests pass       | 15         | pytest -q -> 73 passed    |
| 2  | Coverage >= 80%      | 10         | 96% achieved              |
| 3  | REQ-007 endpoint     | 15         | curl returns 200 + stats  |
| 4  | Docker health check  | 15         | docker inspect -> healthy |
| 5  | Governance audit log | 10         | /audit returns entries    |
| 6  | ADR-002              | 10         | docs/adr/002-*.md         |
| 7  | metrics-report.md    | 10         | REQ-007 row present       |
| 8  | Pre-commit hook      | 10         | hook fires + audit entry  |
| 9  | Submission notes     | 5          | REPORT.md complete        |
|    | <b>Subtotal</b>      | <b>100</b> |                           |
| 10 | Bonus: ADR-003       | +5         | docs/adr/003-*.md         |
| 11 | Bonus: mem_limit     | +5         | docker-compose.yml        |
|    | <b>Total</b>         | <b>110</b> |                           |

### Grader verification commands:

```
cd week-7/capstone-py
pytest -q # 73 passed
pytest --cov=src --cov-report=term-missing # 96% coverage
curl http://localhost:5000/api/notifications/stats # 200 + stats
docker compose up --build -d
docker inspect capstone-notification-service --format='{{.State.Health.Status}}'
curl http://localhost:5000/api/notifications/audit
ls docs/adr/002-stats-computation-strategy.md
ls docs/adr/003-retry-history-on-runtime-error.md
grep mem_limit docker-compose.yml
ls .git/hooks/pre-commit
cat week-7/capstone-py/.audit-log.jsonl
```